Malware Lab

Lab Set Up :

Download the ZIP folder:
https://drive.google.com/file/d/1nEHW-kZ1nnrKSsSvXaw6WeksSemcGJ4k/view?usp=sharing

Unzip the folder; you will see docker.yaml file

Build and Run Containers
docker-compose up --build

List Containers
docker ps

Open a new terminal and access the Attacker Container
docker exec -it attacker_machine bash Or use docksh <container-id>

Open another terminal and access the Target Machine_1 Container
docker exec -it target_machine_1 bash Or use docksh <container-id>

Open another terminal and access the Target Machine_2 Container
docker exec -it target_machine_2 bash Or use docksh <container-id>

Open another terminal and run the following commands:
#docker cp FooVirus.py attacker_machine:/var/
#docker cp AbraWorm.py attacker_machine:/var/

In Attacker Container do the following:

Note down the ip address of attacker
#ifconfig
Create foo files
#echo “hello world” > test1.foo
#echo “bye bye world”> test2.foo
Change the directory
#cd /home/seed
#echo “test”>fun.foo
Come back to var folder
#ls

In Target Machine_1 and Target Machine_2 Container do the following:

Note down the ip address

#ifconfig

Move to /home/seed

#cd /home/seed

#echo “hello world” > test1_target.foo

#chmod 777 test1_target.foo

#echo “bye bye world”> test2_target.foo

#chmod 777 test2_target.foo

Create a new directory

#mkdir new

#echo “test”>fun_target.foo

#chmod 777 fun_target.foo

Part 1 : To learn how malware spreads by first analyzing a virus that infects files locally and then modifying it to create a worm that spreads across machines automatically.

Task 1 Objective :

- Understand how a **file-infecting virus works**
- Analyze the working of FooVirus.py

What is FooVirus.py?

A **self-replicating program** that:

- Finds .foo files
- Copies its own code into them
- Disables original content
- Key Concepts
    - Self-replication
    - File infection
    - No network propagation
    - Local spreading only

Execution Flow (Diagram)

START
↓
Read own code (first 37 lines)
↓
Find all *.foo files
↓
For each file:
↓
Check if already infected
↓
Change permissions
↓
Write virus code at top
↓
Comment original content
↓
END

In Attacker Container do the following:

Observe the content of foo files in attacker container before and after running the following command :
#python3 FooVirus.py

Task 1.1 : Check if the content of fun.foo in /home/seed changed or not after you executed the above command. If not, why?

Now, make the required changes in FooVirus.py such that the foo files across any folder in the attacker container are infected.

Answer the following Questions :

1. Why does the virus read sys.argv[0]?
2. Why only first 37 lines are copied?
3. What happens if reinfection check is removed?
4. Why is original code commented instead of deleted?
5. What type of malware is this and why?

All outputs must be explained with screenshots.

Task 2 Objective :

A virus spreads by infecting files locally, while a worm spreads automatically across systems using network communication. FooVirus.py only spreads locally. To become a worm, it must:

- Spread across machines
- Use network communication

In order to do that, take cues from the code provided for AbraWorm.py, turn the FooVirus.py virus into a worm by incorporating networking code in it and call it FooWorm.py. Paste the FooWorm.py code in the report with explanations and required screenshots.

Worm Flow (Diagram)

ATTACKER MACHINE
↓
Read own code
↓
Connect to remote machine (SSH)
↓
Traverse directories
↓
Download file → /tmp
↓
Infect locally
↓
Upload back
↓
Delete temp file
↓
Repeat

The resulting worm will still infect only the '.foo' files, but it will also have the ability to hop into other machines.

You must show both folder to folder propagation (on the same machine) and container to container propagation during the demo.

Answer the following Questions :

1. Why can't we directly modify remote files?
2. Why is /tmp used in worm design?
3. What is the role of SSH in propagation?
4. How is worm different from virus in this lab?
5. What happens if /tmp file is not deleted?
6. What if wrong credentials are used?
7. How can recursion affect performance?
8. How would you prevent reinfection in worm?
9. How can this worm be made stealthier?
10. What are the limitations of this worm?

Part 2: Modify the AbraWorm.py to make it polymorphic, meaning each copy looks different while performing the same actions.

AbraWorm.py is a simple educational worm that connects to remote machines using SSH by trying username–password combinations. Once it gains access, it searches for files containing a specific keyword ("abracadabra") and downloads them to the local machine. It then copies itself to the target system, allowing the worm to spread. This process demonstrates basic concepts of propagation and data access in a controlled lab environment.

A worm is a type of malware that:

- Spreads automatically across machines
- Copies itself to other systems
- May steal or manipulate data

The given program AbraWorm.py:

1. Connects to a target machine using SSH
2. Searches for files containing "abracadabra"
3. Downloads those files
4. Copies itself to the target machine

You will modify AbraWorm.py so that:
No two copies of the worm are exactly the same on different machines.
Modify the docker files and create another victim container for this task so that you can demonstrate the working of Polymorphic AbraWorm.py

A polymorphic worm changes its code every time it spreads Even though it behaves the same, its appearance (code) changes. This is done to avoid detection and to make signature-based detection difficult

Task 1 : To run AbraWorm.py

First inspect the existence of secret files on Target_Machine_1 and Target_Machine_2. Inspect the folder /home/seed on the target machine containers.

Now go to Attacker Container and run :
$cd var
$python3 AbraWorm.py

Then inspect the stolen folder in var directory of Attacker container
$cd stolen
(Provide screenshot)

Move to Target Machines and check if copy of AbraWorm.py is present.
(Provide Screenshot)

Task 2 : Modify the worm such that before copying itself to the target:

- The code is altered dynamically
- A modified version is sent instead of the original file

Required Modifications: Implement ANY TWO of the following :

1. Add Random Comments
   Example: # random comment: xYz12AbCd
2. Insert Random Blank Lines
   Add multiple \n at random positions
3. Modify Spacing
   Change indentation or spaces between variables
4. Add Dummy Code
   Example: temp = 5 + 10
5. Add Random Print Statements
   Example: print("Hello123")

Show that the version of AbraWorm.py on target_machine_1 and target_machine_2 is different. Provide code explanation and screenshots wherever necessary.

Submission:

All outputs must be explained with respective screenshots.